

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of THURSDAY 8 JUNE 2023 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

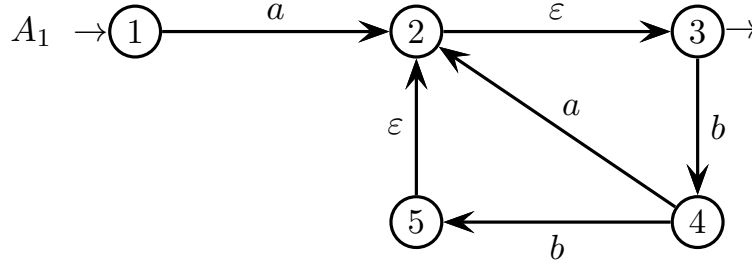
TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): syntax and semantics of languages, divided in four sections:
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic finite-state automaton A_1 below, with spontaneous transitions (ε -transitions), over the two-letter alphabet $\Sigma = \{ a, b \}$:

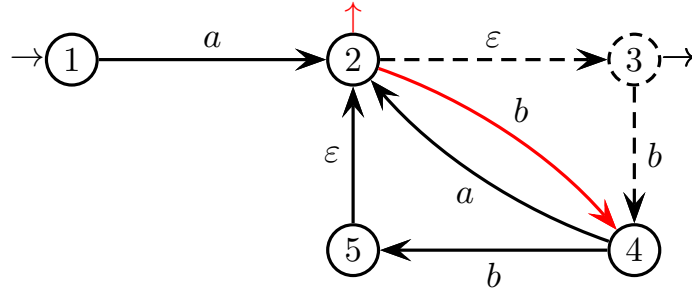


Answer the following questions:

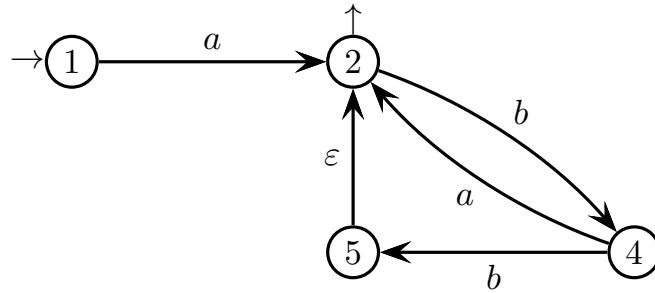
- List all the short strings of language $L(A_1)$ that are of length ≤ 3 , i.e., from length 0 to length 3 included.
 - Cut all the spontaneous transitions of automaton A_1 and so obtain an equivalent automaton A_2 , still possibly nondeterministic. Test the correctness of A_2 with the short valid strings found at point (a), i.e., write their computations.
 - Through the Berry-Sethi (BS) method, from automaton A_1 obtain an equivalent deterministic automaton A_3 . Test the correctness of A_3 with the short valid strings found at point (a), i.e., write their computations.
 - Check if automaton A_3 is minimal, and if necessary minimize it.
 - Through the Brzozowski-McCluskey (BMC) method, from automaton A_3 obtain an equivalent regular expression R_1 . Test the correctness of R_1 with the short valid strings found at point (a), i.e., write their derivations.
 - (optional) Write a regular expression R_1^R that generates the language $L(R_1)^R$, i.e., the mirror image of language $L(R_1)$.
-

Solution

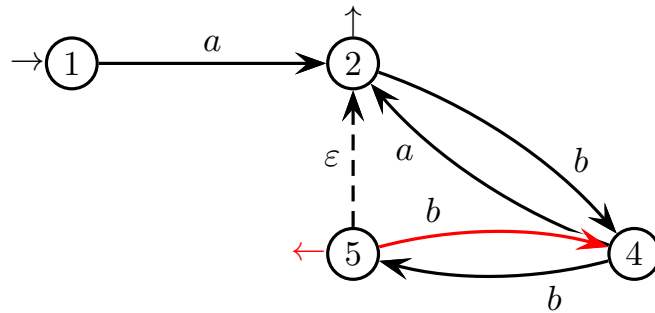
- (a) Here are the valid strings of $L(A_1)$ with length ≤ 3 : a , aba and abb .
- (b) Cut of the ε -transitions of automaton A_1 through backward propagation.
Cut transition $2 \xrightarrow{\varepsilon} 3$ (the red elements are the back-propagated ones):



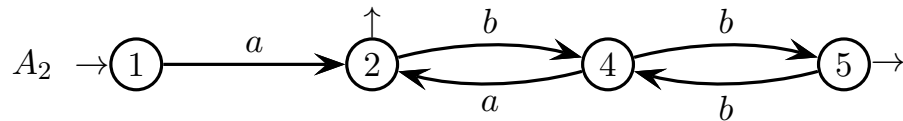
State 3 becomes unreachable and the automaton can be cleaned by removing the dashed elements:



Cut transition $5 \xrightarrow{\varepsilon} 2$ (the red elements are the back-propagated ones):

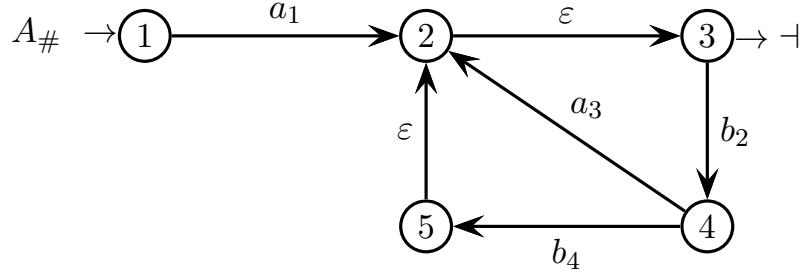


Finally (by removing the dashed elements and reshaping the graph):

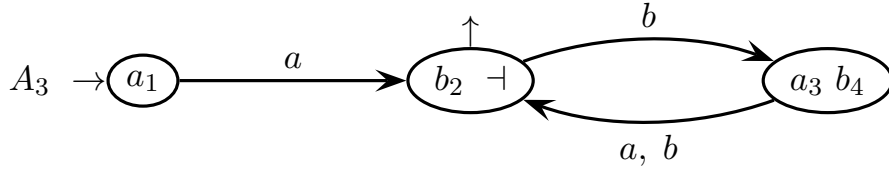


Automaton A_2 is (incidentally) deterministic. It is correct as it accepts all the short strings of point (a) and no other strings of length ≤ 3 . Equivalent automata may be obtained, deterministic or not, and even with fewer states. Writing the computations of the short strings is left to the reader.

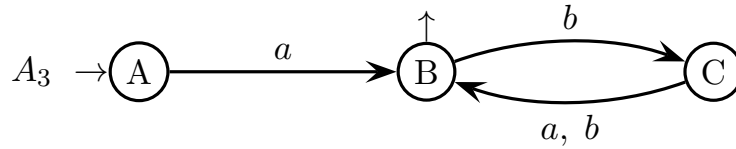
(c) Berry-Sethi method for a det. automaton A_3 equivalent to automaton A_1 :



initials	a_1
terminal	followers
a_1	$b_2 \dashv$
b_2	$a_3 \ b_4$
a_3	$b_2 \dashv$
b_4	$b_2 \dashv$



With a change of state names, automaton A_3 becomes:

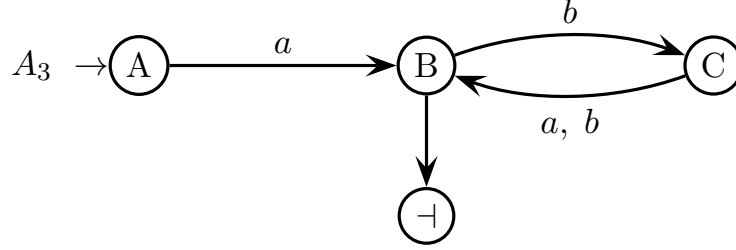


The correctness test of automaton A_3 (computations) is left to the reader.

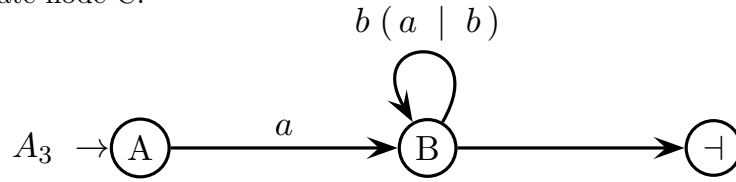
- (d) The deterministic automaton A_3 is already in clean form. The two non-final states A and C of A_3 are distinguishable, as state C lacks the b -transition. The final state B is unique. Therefore automaton A_3 is even already minimal.

Note that automaton A_2 is deterministic, too (thus minimization theory is applicable to it), and that its states 2 and 5 are undistinguishable. By merging such states, automaton A_2 becomes minimal and of course identical to automaton A_3 (except for the state names).

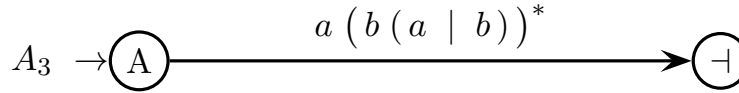
- (e) Automaton A_3 has to be normalized with a final state with no outgoing arc (while the initial state is already non-reentering), as follows:



Eliminate node C:



Eliminate node B:



Finally:

$$R_1 = a (b (a \mid b))^*$$

As for the derivations of the valid short strings:

$$R_1 \xrightarrow{\text{choose } * = 0} a \varepsilon = a$$

$$R_1 \xrightarrow{\text{choose } * = 1} a b (a \mid b) \xrightarrow{\text{choose left member of } |} a b a$$

$$R_1 \xrightarrow{\text{choose } * = 1} a b (a \mid b) \xrightarrow{\text{choose right member of } |} a b b$$

Thus expression R_1 successfully passes the correctness test.

- (f) Here is the mirror regular expression R_1^R :

$$R_1^R = ((b \mid a) b)^* a$$

Of course, writing $b \mid a$ rather than $a \mid b$ is equivalent, as union is commutative (choosing which form should be used is only a matter of style).

2 Free Grammars and Pushdown Automata 20%

1. A series of BNF grammars is given below. Answer the following questions:

(a) Consider the following grammar G_1 (axiom S):

$$G_1: S \rightarrow a S b \mid S b \mid \varepsilon$$

Argue that grammar G_1 is ambiguous by providing the shortest ambiguous string and drawing all its syntax trees.

(b) Write a non-ambiguous BNF grammar G_2 equivalent to grammar G_1 .

(c) Consider next the following grammar G_3 (axiom S):

$$G_3 \begin{cases} S \rightarrow A B \\ A \rightarrow a A b \mid A b \mid \varepsilon \\ B \rightarrow b B c \mid b B \mid \varepsilon \end{cases}$$

Argue that grammar G_3 is ambiguous by drawing all the (three) syntax trees for the valid string:

bb

(d) Write a non-ambiguous BNF grammar G_4 equivalent to grammar G_3 .

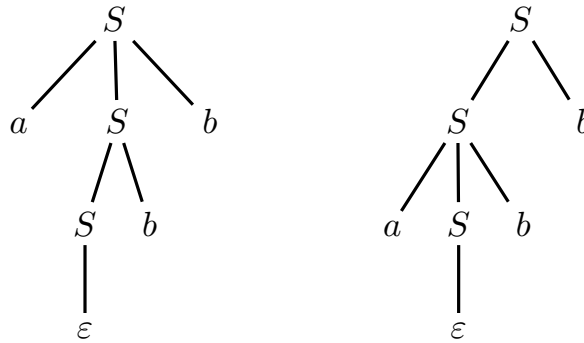
(e) (optional) Draw the syntax tree for the valid string:

$abbb c$

according to grammar G_4 .

Solution

- (a) Grammar G_1 is ambiguous, as the order of use of the non-null alternative rules for the axiom S is free. Here are the shortest ambiguous string and all its syntax trees (two):

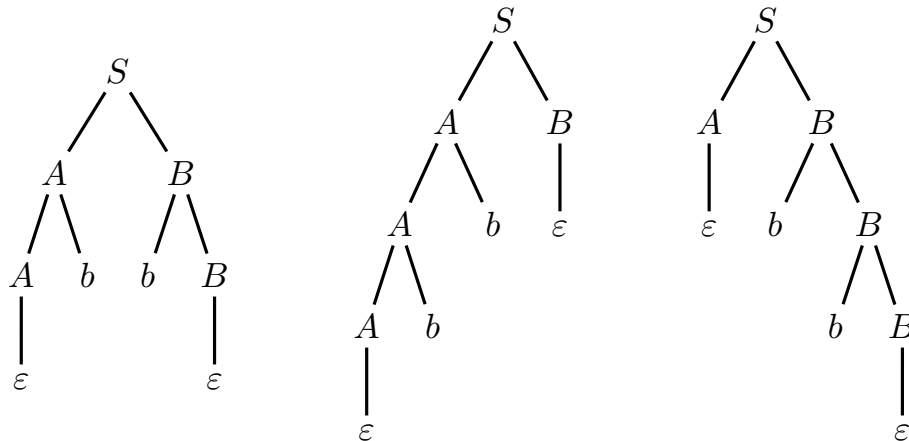


- (b) Here is a non-ambiguous grammar G_2 (axiom S) equivalent to grammar G_1 :

$$G_2 \left\{ \begin{array}{l} S \rightarrow a S b \mid X \\ X \rightarrow X b \mid \varepsilon \end{array} \right.$$

In grammar G_2 , the order of use of the non-null alternative rules is fixed, by introducing an additional nonterminal X .

- (c) Grammar G_3 is ambiguous. Here are all the syntax trees for string bb (three):



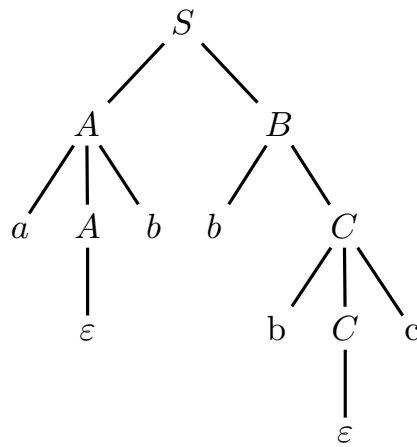
Grammar G_3 suffers from concatenation ambiguity, due to the unbalanced letters b , which can be generated by both nonterminals A and B .

(d) Here is a non-ambiguous grammar G_4 (axiom S) equivalent to grammar G_3 :

$$G_4 \left\{ \begin{array}{l} S \rightarrow A B \\ A \rightarrow a A b \mid \varepsilon \\ B \rightarrow b B \mid C \\ C \rightarrow b C c \mid \varepsilon \end{array} \right.$$

In grammar G_4 , ambiguity is eliminated by generating all the unbalanced letters b only through nonterminal B .

(e) Here is the syntax tree for the string $abbbbc$ according to grammar G_4 :



The unique unbalanced letter b can be generated only by nonterminal B .

2. Consider (a simplified version of) the $\text{\LaTeX}$ language for the definition of text documents. Every document contains a preamble that specifies the document class, e.g., article, book, ... (the class name is a text string), and that includes zero or more packages. The preamble is followed by a document block.

Every package is specified with a command `\usepackage[options]{package_name}`. The options can be omitted, along with the surrounding square brackets. The square brackets can contain one or more options, separated by commas. An option can be an identifier or a pair identifier/value, where the identifier and the value are separated by a sign “=”, and the value is a text string. The package name is an identifier.

The document block is introduced by the clause `\begin{document}` and is concluded by the clause `\end{document}`. The block (possibly empty) contains text strings possibly interspersed with environments.

Every environment is delimited by the clauses `\begin{env_name}` and `\end{env_name}`, where the environment name is an identifier. An environment (possibly empty) contains text strings as well as other environments. Note that since the number of environments is arbitrary, the equality of the environment names in a pair of clauses `\begin{env_name}` and `\end{env_name}` cannot be modeled syntactically.

```
\documentclass{article}

\usepackage[a4paper, margin=1cm]{geometry}
\usepackage[disable]{todonotes}
\usepackage{comment}

\begin{document}
  \begin{comment}
    commented text including also another environment
    \begin{comment}
      a nested comment
    \end{comment}
  \end{comment}
  more arbitrary text and commands could follow
\end{document}
```

Answer the following questions:

- (a) Write an EBNF grammar for the $\text{\LaTeX}$ subset described above, considering the nonterminal `STRING` as already defined to generate any text string, and the terminal `id` as representing any identifier. Spaces can be safely ignored.
- (b) Draw the abstract syntax tree of the sample document below:

```
\documentclass{article}

\usepackage[disable]{todonotes}
\usepackage{comment}

\begin{document}
  text
\end{document}
```

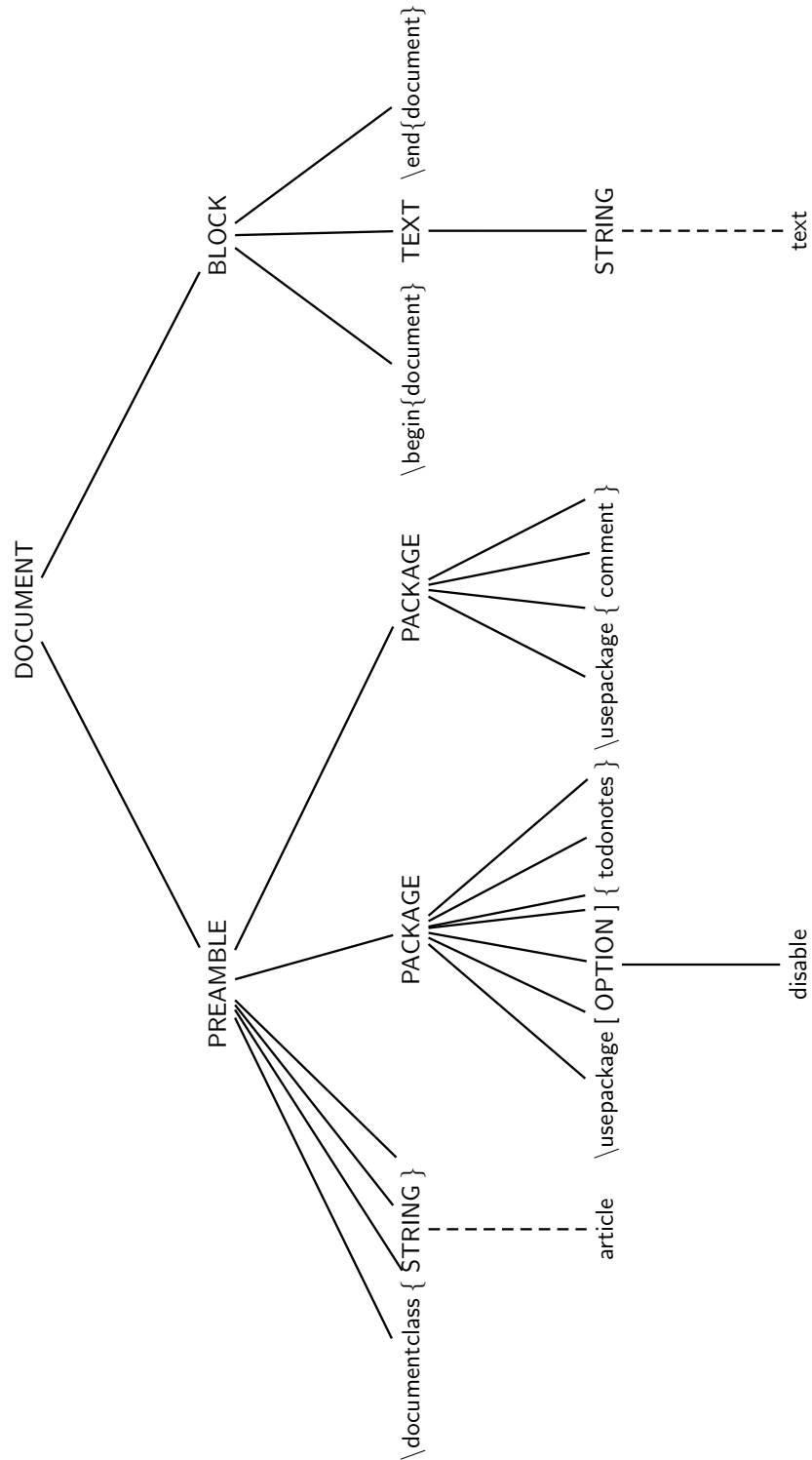
Solution

(a) Here is a possible EBNF grammar (axiom DOCUMENT), with layered rules:

$$\left\{ \begin{array}{l}
 \langle \text{DOCUMENT} \rangle \rightarrow \langle \text{PREAMBLE} \rangle \langle \text{BLOCK} \rangle \\
 \hline
 \langle \text{PREAMBLE} \rangle \rightarrow \backslash \text{documentclass} \{ \\
 \qquad \qquad \langle \text{STRING} \rangle \\
 \qquad \qquad \} \langle \text{PACKAGE} \rangle^* \\
 \langle \text{BLOCK} \rangle \rightarrow \backslash \text{begin}\{\text{document}\} \\
 \qquad \qquad \langle \text{TEXT} \rangle \\
 \qquad \qquad \backslash \text{end}\{\text{document}\} \\
 \hline
 \langle \text{PACKAGE} \rangle \rightarrow \backslash \text{usepackage} [\\
 \qquad \qquad \qquad \qquad \text{"["} \langle \text{OPTION} \rangle (, \langle \text{OPTION} \rangle)^* \text{"}] \\
 \qquad \qquad \qquad \qquad] \{ \text{id} \} \\
 \langle \text{TEXT} \rangle \rightarrow (\langle \text{STRING} \rangle \mid \langle \text{ENVIRON} \rangle)^* \\
 \hline
 \langle \text{OPTION} \rangle \rightarrow \text{id} [= \langle \text{STRING} \rangle] \\
 \langle \text{ENVIRON} \rangle \rightarrow \backslash \text{begin}\{\text{id}\} \langle \text{TEXT} \rangle \backslash \text{end}\{\text{id}\} \\
 \hline
 \dots\dots\dots
 \end{array} \right.$$

As usual, the square brackets indicate optionality. The square brackets enclosed between apices are instead used as grammar terminals. Nonterminal STRING is expanded by rules not shown here.

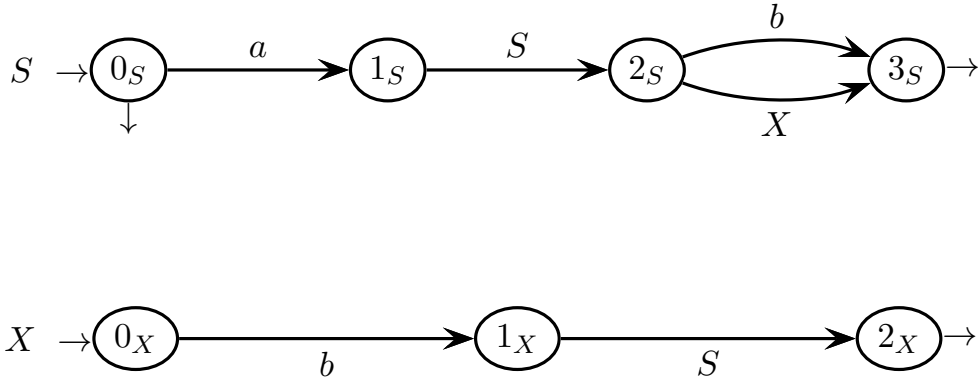
(b) Syntax tree of the sample document (drawn according to the grammar):



Note that nonterminal **STRING** is here expanded to the text string it generates. The subtree of **STRING** is just represented as a dashed edge. Terminal **id** instead is just replaced by the actual identifier that occurs in the sample text.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented as machine net, over the two-letter terminal alphabet $\Sigma = \{ a, b \}$ and the two-symbol nonterminal alphabet $V = \{ S, X \}$ (axiom S):

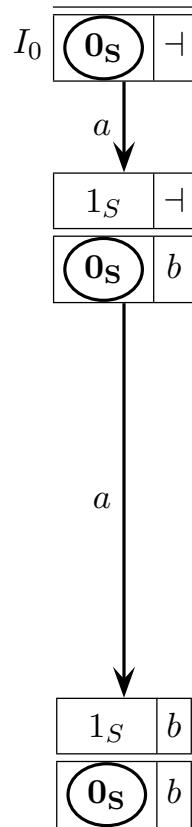


Answer the following questions (use the figures / tables / spaces on the next pages):

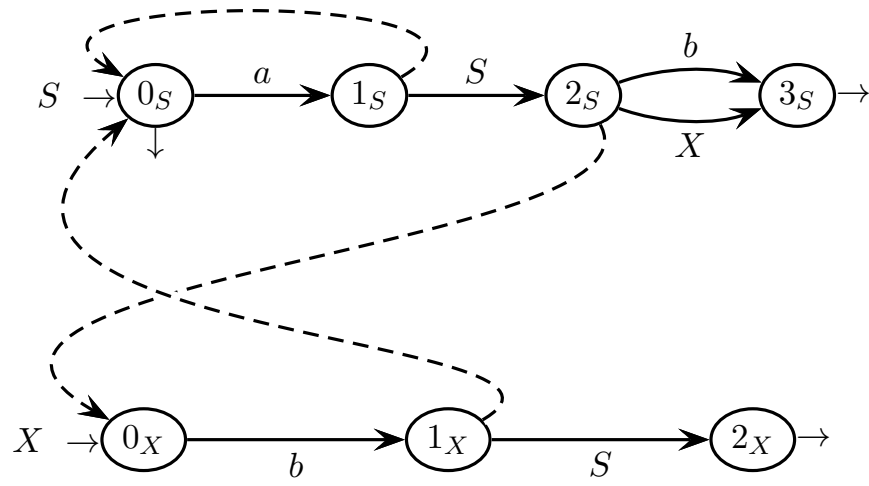
- (a) List the four shortest strings that are generated by grammar G , and show a syntax tree for each of them.
- (b) Draw the complete ELR pilot of grammar G , examine all the possible conflicts (if any), and say if grammar G is of type ELR (1) or not.
- (c) Find all the guide sets for the machine net of grammar G , with $k = 1$, examine all the possible conflicts (if any), and say if grammar G is of type ELL (1) or not.
- (d) Is grammar G ambiguous or not ? Justify your answer.
- (e) (optional) Is language $L(G)$ inherently ambiguous or not ? Justify your answer.

space for question (a) – short strings and their trees

space for question (b) – complete ELR pilot and conflicts



space for question (c) – guide sets and conflicts

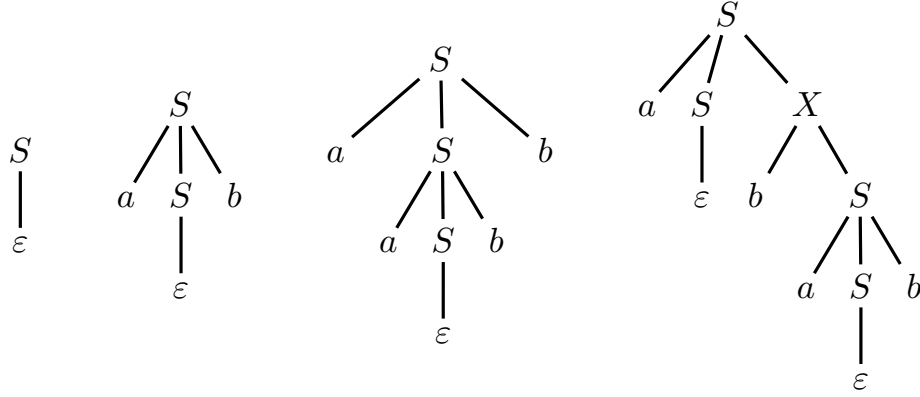


space for question (d) – ambiguity of the grammar

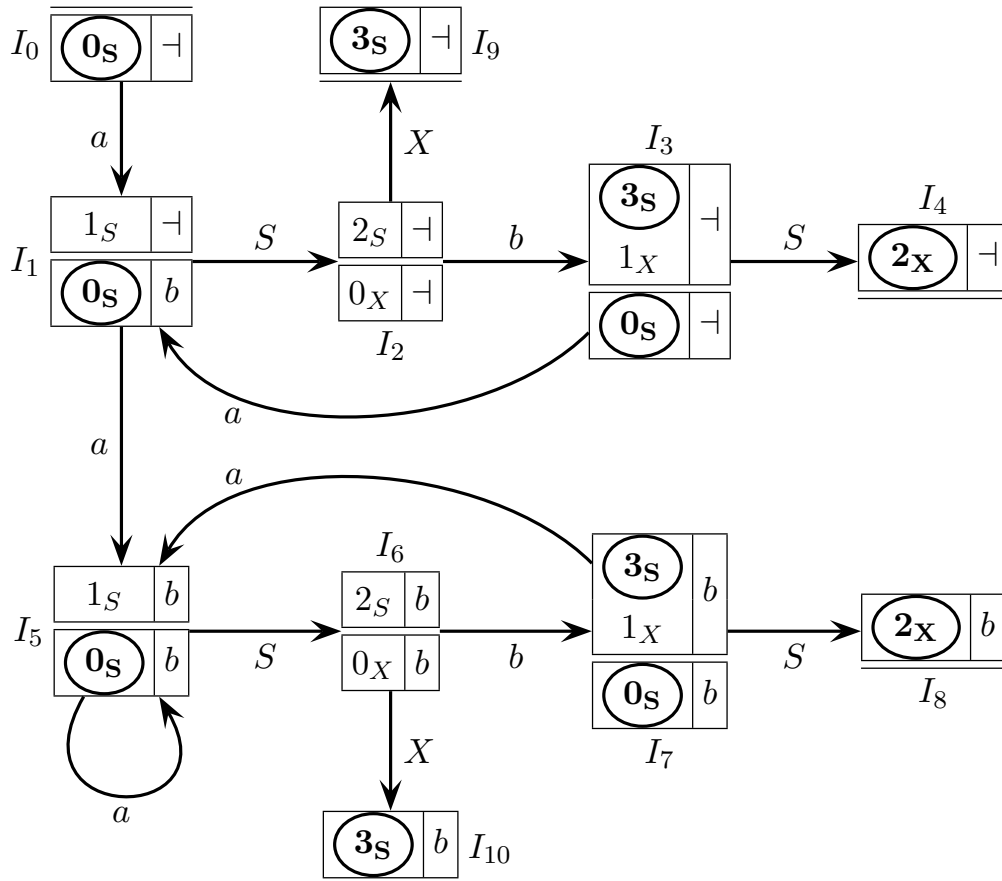
space for question (e) – inherent ambiguity of the language

Solution

(a) Here are the four shortest strings: ε , ab , $abbb$ and $abab$. Their trees:

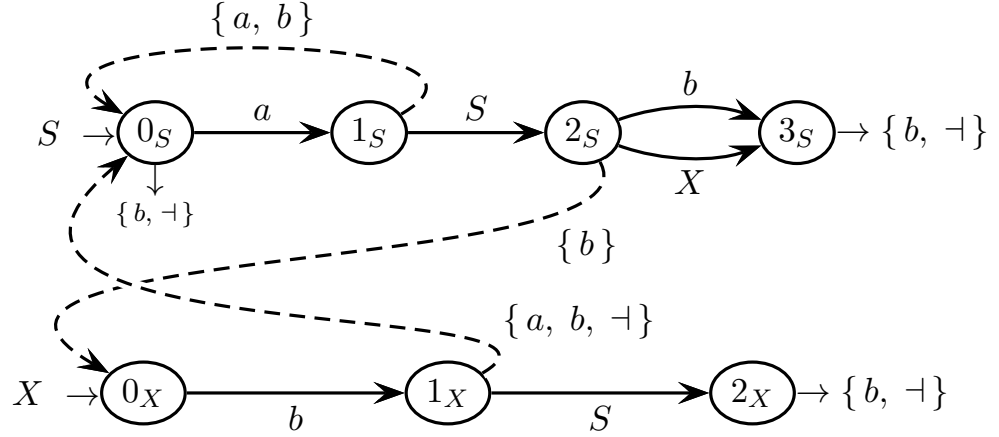


(b) Here is the complete ELR pilot of grammar G , with eleven m-states:



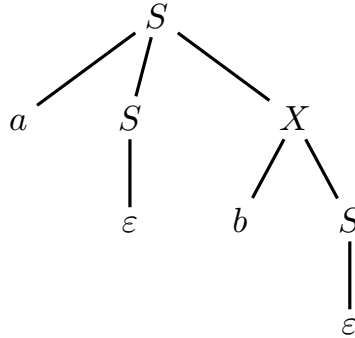
The pilot has two RR conflicts on $\neg$ and b in the m-states I_3 and I_7 , respectively. There are two multiple (double) transitions, namely $I_2 \xrightarrow{b} I_3$ and $I_6 \xrightarrow{b} I_7$, but neither is convergent, so there are no Convergence conflicts. Anyway, grammar G has conflicts, therefore it is not of type ELR(1). Note that the pilot does not have the STP either.

(c) Here is the PCFG of grammar G , with all the call arcs and guide sets:



The PCFG has one conflict on b in the state 2_S . Therefore grammar G is not of type ELL (1). This was expected, as grammar G is not of type ELR (1) either.

(d) Yes, grammar G is ambiguous. The valid string ab has two syntax trees. One (the simpler) is already shown at point (a), and the other one is as follows:



Also the valid strings $abbb$ and $abab$ are ambiguous, of course.

(e) From the four shortest strings of point (a), which are exactly the Dyck strings of length ≤ 4 with a and b open and closed brackets, respectively, one might conjecture that language $L(G)$ is Dyck. This turns out to be true. In fact, the net can be rewritten as two rules:

$$S \rightarrow a S (b \mid X) \mid \varepsilon \quad X \rightarrow b S$$

then by substitution of nonterminal X :

$$S \rightarrow a S (b \mid b S) \mid \varepsilon$$

that is, by factoring letter b :

$$S \rightarrow a S b (\varepsilon \mid S) \mid \varepsilon$$

and since the axiom S is nullable (hence $\varepsilon \mid S$ is equivalent to S):

$$S \rightarrow a S b S \mid \varepsilon$$

which is a well-known (non-ambiguous) grammar of Dyck. Therefore language $L(G)$ coincides with Dyck and is not inherently ambiguous.

4 Language Translation and Semantic Analysis 20%

1. Consider the following source grammar G^s (axiom S):

$$G^s \left\{ \begin{array}{l} S \rightarrow a S b \\ S \rightarrow T \\ T \rightarrow c T d \\ T \rightarrow \varepsilon \end{array} \right.$$

which generates the source language L^s below:

$$L^s = \{ a^h c^k d^k b^h \mid h, k \geq 0 \}$$

Answer the following questions:

- (a) Define a translation grammar G_1^T for the translation τ_1 defined as follows on the source language L^s generated by grammar G^s :

$$\tau_1(a^h c^k d^k b^h) = b^h d^k c^k a^h$$

- (b) Say if the inverse translation τ_1^{-1} is a function. In the positive case, provide a translation grammar $G_1^{T^{-1}}$ for τ_1^{-1} .
- (c) Consider now the translation τ_2 defined as follows on the source language L^s generated by grammar G^s :

$$\tau_2(a^h c^k d^k b^h) = \begin{cases} b^h c^k d^k a^h & \text{if } k \text{ is even} \\ b^h d^k c^k a^h & \text{if } k \text{ is odd} \end{cases}$$

By suitably modifying the source grammar G^s , provide a translation grammar G_2^T that defines translation τ_2 .

- (d) Draw the translation syntax tree for the sample translation τ_2 below, according to grammar G_2^T :

$$\tau_2(a c d b) = b d c a$$

Solution

(a) Here is the translation grammar G_1^τ for τ_1 (axiom S):

$$G_1^\tau \left\{ \begin{array}{l} S \rightarrow \frac{a}{b} S \frac{b}{a} \\ S \rightarrow T \\ T \rightarrow \frac{c}{d} T \frac{d}{c} \\ T \rightarrow \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

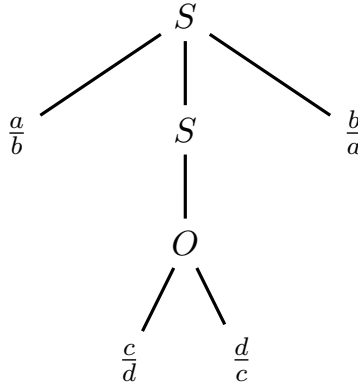
(b) Translation τ_1 simply computes the mirror image of the source string, i.e., $\tau_1(x) = x^R$, and clearly is a function. The inverse function of mirroring is still mirroring, since $(x^R)^R = x$. Thus the inverse translation is $\tau_1^{-1}(x) = x^R$, and is a function as well. Therefore, a translation grammar $G_1^{\tau^{-1}}$ for τ_1^{-1} is the one where the source and target parts of grammar G_1^τ are switched:

$$G_1^{\tau^{-1}} \left\{ \begin{array}{l} S \rightarrow \frac{b}{a} S \frac{a}{b} \\ S \rightarrow T \\ T \rightarrow \frac{d}{c} T \frac{c}{d} \\ T \rightarrow \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

(c) Here is the (modified) translation grammar G_2^τ for τ_2 (axiom S):

$$G_2^\tau \left\{ \begin{array}{l} S \rightarrow \frac{a}{b} S \frac{b}{a} \\ S \rightarrow E \\ S \rightarrow O \\ E \rightarrow \frac{c}{c} \frac{c}{c} E \frac{d}{d} \frac{d}{d} \\ E \rightarrow \frac{\varepsilon}{\varepsilon} \\ O \rightarrow \frac{c}{d} \frac{c}{d} O \frac{d}{c} \frac{d}{c} \\ O \rightarrow \frac{c}{d} \frac{d}{c} \end{array} \right. \quad \text{compacted} \quad G_2^\tau \left\{ \begin{array}{l} S \rightarrow \frac{a}{b} S \frac{b}{a} \\ S \rightarrow E \\ S \rightarrow O \\ E \rightarrow \frac{cc}{cc} E \frac{dd}{dd} \\ E \rightarrow \frac{\varepsilon}{\varepsilon} \\ O \rightarrow \frac{cd}{dd} O \frac{dc}{cc} \\ O \rightarrow \frac{cd}{dc} \end{array} \right.$$

(d) Here is the translation tree of the translation sample for τ_2 :



The tree is drawn according to the non-compacted form of grammar G_2^τ .

2. Consider the grammar below, which represents a (non-empty) list of names separated by commas (axiom S):

$$\left\{ \begin{array}{l} 1: S \rightarrow L \\ 2: L \rightarrow n \ c \ L \\ 3: L \rightarrow n \end{array} \right.$$

where symbol n represents an alphabetical string terminal and symbol c is a comma.

A valid string generated by the grammar could be:

Bianchi, Rossi, Verdi

We want to define an attribute grammar that computes whether the names are listed in alphabetical order.

Answer the following questions (use the tables / trees / spaces on the next pages):

- (a) We want to check if the names are listed in alphabetical order. To this end, we define two attributes f and o , as follows:
- Attribute f is synthesized (left) and of type string. It represents the string corresponding to the first name in a list. To initialize this attribute, consider the availability of a function *string* that, when applied to a terminal, returns the associated text in the parsed string.
 - Attribute o is synthesized (left) and of type boolean. It represents whether the names in a list are ordered (*true*) or not (*false*).

Write the semantic functions for computing the two attributes. You can assume that strings can be compared alphabetically by means of the usual relational operator “ $<$ ”. The final result is provided by attribute o in the root node S .

- (b) Are the two attributes f and o computable in the one-sweep way ?
- (c) (optional) Write the procedure of the integrated syntactic-semantic recursive descent analyzer for nonterminal L , according to the rule below:

$$L \rightarrow n \ c \ L \mid n$$

table to be filled in for question (a)

#	<i>syntax</i>	<i>semantic functions</i>
---	---------------	---------------------------

1:	$S_0 \rightarrow L_1$	
----	-----------------------	--

2:	$L_0 \rightarrow n \ c \ L_1$	
----	-------------------------------	--

3:	$L_0 \rightarrow n$	
----	---------------------	--

space for answering question (b)

space for answering question (c)

Solution

(a) Here is the attribute grammar:

#	<i>syntax</i>	<i>semantic functions</i>
1:	$S_0 \rightarrow L_1$	$o_0 := o_1$
2:	$L_0 \rightarrow n \ c \ L_1$	$f_0 := \text{string}(n)$ if $((f_0 < f_1) \text{ and } (o_1 = \text{true}))$ then $o_0 := \text{true}$ else $o_0 := \text{false}$ endif
3:	$L_0 \rightarrow n$	$f_0 := \text{string}(n)$ $o_0 := \text{true}$

(b) The attribute grammar is purely synthesized. Thus both attributes f and o can be computed in the one-sweep way (in particular, bottom-up), as they are of type left and there are no attributes of type right.

- (c) From point (b), we know that the attribute grammar is one-sweep. Yet the syntactic support is not deterministic LL (1), since for a look-ahead window of size $k = 1$, both the alternative rules 2: $L \rightarrow n c L$ and 3: $L \rightarrow n$ of nonterminal L have the same guide set $\{n\}$. However, the support is deterministic LL (2), since for a window of size $k = 2$, the alternative rules have a guide set $\{nc\}$ and $\{n \dashv\}$, respectively. Furthermore, the sibling graph of alternative 2 is empty, as there are no right attributes. Thus there exists an integrated syntactic-semantic recursive descent analyzer. Here is the procedure of L , with $k = 2$:

```

procedure  $L$  (out:  $f_0, o_0$ )
  var  $f_1, o_1, name$ 
  // window  $\langle cc1, cc2 \rangle$  is expected to be either  $\langle n, c \rangle$  or  $\langle n, \dashv \rangle$ 
  if  $cc1 = n$  then
     $name := string(cc1)$  // stores current input text
    switch  $cc2$  do // chooses alternative 2 or 3
      case  $c$  do // alternative 2:  $L \rightarrow n c L$ 
        shift // window becomes  $\langle c, x \rangle$ 
        shift // window becomes  $\langle x, y \rangle$ 
        if  $cc1 = n$  then
          call  $L(f_1, o_1)$ 
           $f_0 := name$ 
          if  $((f_0 < f_1) \text{ and } (o_1 = true))$  then  $o_0 := true$ 
          else  $o_0 := false$ 
          if  $cc1 = \dashv$  then return
          else error
        else error
      case  $\dashv$  do // alternative 3:  $L \rightarrow n$ 
        shift // window becomes  $\langle \dashv, \dashv \rangle$ 
         $f_0 := name$ 
         $o_0 := true$ 
        if  $cc1 = \dashv$  then return
        else error
      else error
    else error
  end procedure

```

The look-ahead window (of size $k = 2$) is $\langle cc1, cc2 \rangle$: the global variables $cc1$ and $cc2$ represent the current and next input symbols, respectively. Thus $cc2$ has to be used to choose the alternative 2 or 3 (**switch** clause). The auxiliary function **shift** moves the window one position forward, e.g., moves $\langle n, c \rangle$ to $\langle c, x \rangle$ where x is a new input symbol (or is $\dashv$ if the input has terminated), and then to $\langle x, y \rangle$ if it is called again, where y is the second new input.¹

The local variable $name$ temporarily stores the actual input text that corresponds to the grammar terminal n , in order to later assign the left attribute f . In fact, remember that the left attributes are output arguments of the procedure and that they should be assigned immediately before returning from the procedure.

¹In practice, one call to **shift** executes $cc1 := cc2$ and $cc2 := next$.

As it often happens, some checks may be redundant and could be factored or eliminated, and we could cut short on some algorithmic aspects:

```

procedure  $L$  (out:  $f_0, o_0$ )
  var  $f_1, o_1$                                 // suppress variable name
  if  $cc1 = n$  then
     $f_0 := \text{string}(cc1)$                     // anticipate the computation of  $f$ 
    switch  $cc2$  do
      case  $c$  do
        shift shift                            // shift twice
        if  $cc1 = n$  then
          call  $L(f_1, o_1)$ 
           $o_0 := (f_0 < f_1)$  and  $o_1$         // shorten the update of  $o$ 
          if  $cc1 = \neg$  then return
        case  $\neg$  do                            // remove redundancy
           $o_0 := \text{true}$ 
          return
      error                                    // group all errors here
    end procedure

```

The local variable *name* is suppressed by prepending the assignment of attribute *f* directly at the beginning, the code for updating attribute *o* is shortened into one formula, the alternative rule 3 is simplified by removing the redundant shift and return check, and all the error cases are grouped at the end of the procedure. The rest of the code is unchanged.

Even more succinctly, we can anticipate also the assignment of attribute *o*, which will be reassigned later in the alternative rule 2, and we can replace the **switch** clause with a simple concatenated **if** clause, as follows:

```

procedure  $L$  (out:  $f_0, o_0$ )
  var  $f_1, o_1$ 
  if  $cc1 = n$  then
     $f_0 := \text{string}(cc1)$ 
     $o_0 := \text{true}$                                 // may be reassigned later
    if  $cc2 = c$  then
      shift shift
      if  $cc1 = n$  then
        call  $L(f_1, o_1)$ 
         $o_0 := (f_0 < f_1)$  and  $o_1$         // reassignment
        if  $cc1 = \neg$  then return
      else if  $cc2 = \neg$  then return
    error
  end procedure

```

Other optimizations may exist, of course.